Roll Number:-2070120

Name:-Samiksha Sanjay Adake

Assignment Number:-2

Problem Statement:
Design and implement a C++ program to model a banking system where the class BankAccount keeps track of the total number of accounts created and the total balance across all accounts. Use static data members to maintain these values, and static member functions to retrieve and display them. Add a member function in BankAccount class that takes another BankAccount object as an argument and display the balances in ascending order.

Theory:

1. Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm that organizes a program using classes and objects. It combines data and functions into a single unit called a class. OOP makes programs modular, secure, reusable, and easier to maintain. In this program, OOP is implemented using the BankAccount class.

2. Class

A class is a user-defined data type that acts as a blueprint for creating objects. It contains data members and member functions. In this program, BankAccount is a class that stores the account balance and provides functions to display balances and maintain common account information.

3. Object

An object is an instance of a class. Each object has its own copy of non-static data members. In this program, different BankAccount objects represent different bank accounts.

4. Data Members

Data members are variables declared inside a class. They store the information of an object. In this program, balance is a non-static data member, while totalAccounts and totalBalance are static data members.

5. Member Functions

Member functions are functions defined inside a class that operate on the data members. In this program, functions are used to create accounts, display balances, compare balances, and retrieve common account information.

6. Static Data Members

A static data member is shared by all objects of a class. Only one copy exists regardless of how many objects are created. In this program, totalAccounts stores the total number of accounts, and totalBalance stores the sum of balances of all accounts.

7. Static Member Functions

A static member function can access only static members of a class. It can be called using the class name without creating an object. In this program, static functions display the total number of accounts and the total balance.

8. Access Specifiers

Access specifiers control the accessibility of class members. In this program, data members are declared under private to protect the data, while member functions are declared under public so they can be accessed from the main() function.

9. Encapsulation

Encapsulation is the process of combining data members and member functions into a single class while restricting direct access to data. In this program, the BankAccount class encapsulates account information and related operations.

10. Object as Function Argument

An object of a class can be passed as an argument to another member function. In this program, a member function accepts another BankAccount object and compares the balances to display them in ascending order.

11. Comparison

Comparison is performed using relational operators such as <, >, and ==. In this program, the balance of one account is compared with another to arrange them in ascending order.

12. Function

A function is a block of code that performs a specific task. In this program, functions are used to create accounts, compare balances, and display account details.

13. Main Function

The main() function is the starting point of every C++ program. In this program, it creates BankAccount objects, accepts input, calls member functions, and displays the results.

14. Input and Output

The cin object is used to accept input from the user, while cout is used to display the output. In this program, the account balances are entered using cin and displayed using cout.

15. Dot (.) Operator

The dot (.) operator is used to access the public members of an object. In this program, it is used to call member functions such as creating accounts and comparing balances.

Code:

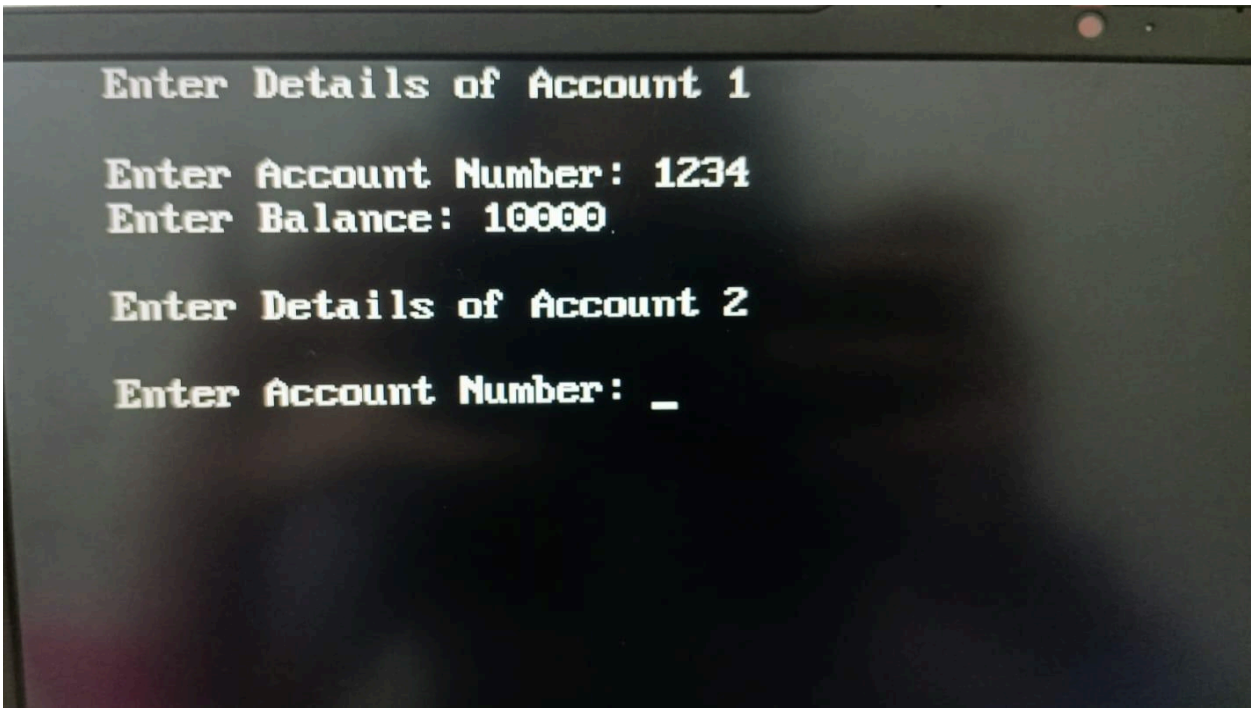
```
#include<iostream.h>
#include<conio.h>

class BankAccount
{
private:
    int accNo;
    float balance;

    static int totalAccounts;
    static float totalBalance;

public:

    void getDetails()
    {
        cout<<"\nEnter Account Number: ";
        cin>>accNo;

        cout<<"Enter Balance: ";
        cin>>balance;

        totalAccounts++;
```

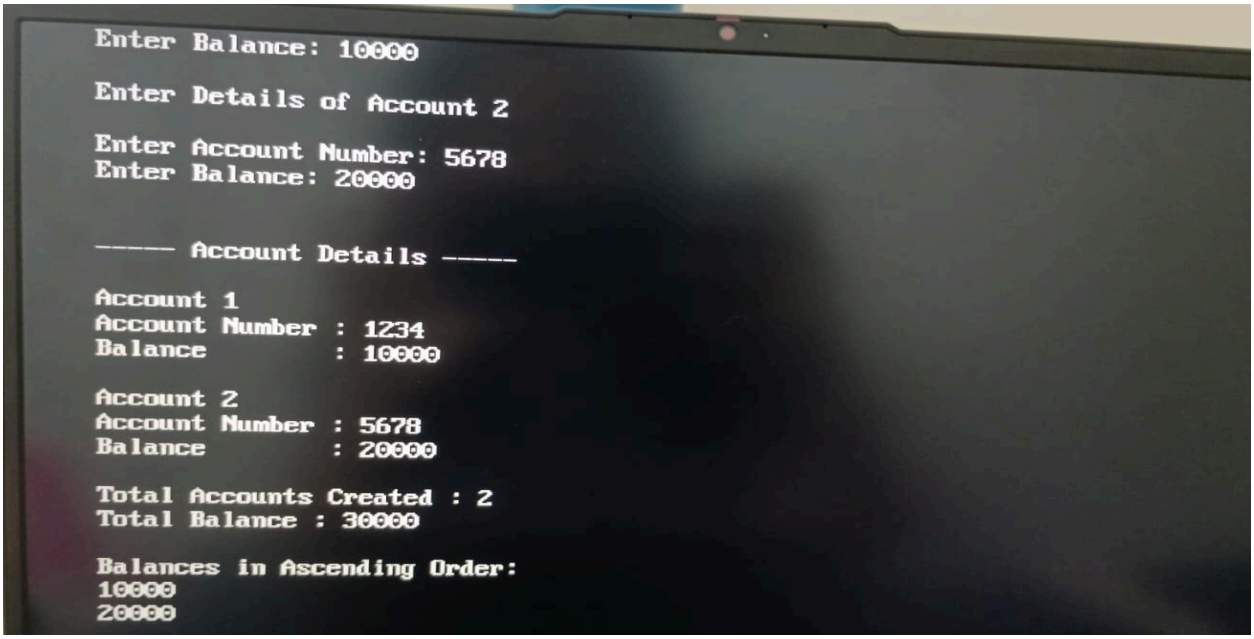
```
    totalBalance = totalBalance + balance;
}


void display()
{
    cout<<"\nAccount Number : "<<accNo;
    cout<<"\nBalance        : "<<balance<<endl;
}


static void showTotalAccounts()
{
    cout<<"\nTotal Accounts Created : "<<totalAccounts;
}


static void showTotalBalance()
{
    cout<<"\nTotal Balance : "<<totalBalance;
}


void displayAscending(BankAccount b)
{
    cout<<"\nBalances in Ascending Order:\n";

    if(balance < b.balance)
```

```
        {
            cout<<balance<<endl;
            cout<<b.balance<<endl;
        }
        else
        {
            cout<<b.balance<<endl;
            cout<<balance<<endl;
        }
    }
};


int BankAccount::totalAccounts = 0;
float BankAccount::totalBalance = 0;

void main()
{
    clrscr();

    BankAccount b1, b2;

    cout<<"Enter Details of Account 1\n";
    b1.getDetails();

    cout<<"\nEnter Details of Account 2\n";
    b2.getDetails();
```

```
    cout<<"\n\n----- Account Details -----";

    cout<<"\n\nAccount 1";
    b1.display();

    cout<<"\nAccount 2";
    b2.display();

    BankAccount::showTotalAccounts();
    BankAccount::showTotalBalance();

    cout<<"\n";
    b1.displayAscending(b2);

    getch();
}
```

Output:


Enter Details of Account 1
Enter Account Number: 1234
Enter Balance: 10000
Enter Details of Account 2
Enter Account Number: _


Enter Balance: 10000
Enter Details of Account 2
Enter Account Number: 5678
Enter Balance: 20000
----- Account Details -----
Account 1
Account Number : 1234
Balance : 10000
Account 2
Account Number : 5678
Balance : 20000
Total Accounts Created : 2
Total Balance : 30000
Balances in Ascending Order:
10000
20000

Outcomes:

- Understand the use of static data members and static member functions.
- Learn how to maintain data that is common to all objects.
- Pass an object as a function argument.
- Compare and display account balances in ascending order.
- Apply the concepts of classes, objects, encapsulation, and static members in C++.